Базы данных и возможности оконных функций

Оконные функции в той или иной степени поддерживаются во всех современных реляционных СУБД.

Задачи, которые непринужденно решаются с помощью оконных функций в SQL:

- Ранжирование (всевозможные рейтинги)
- Сравнение со смещением (соседние элементы и границы)
- Агрегация (сумма и среднее)
- Скользящие агрегаты (сумма и среднее в динамике)

Конечно, это не исчерпывающий список.

Ранжирование

Ранжирование — это всевозможные рейтинги, начиная от призеров чемпионата мира по плаванию и заканчивая Forbes 500.

Мы же будем ранжировать сотрудников в игрушечной таблице employees, вот такой:

id	name	city	department	salary
11	Дарья	Самара	hr	70
12	Борис	Самара	hr	78
21	Елена	Самара	it	84
22	Ксения	Москва	it	90
23	Леонид	Самара	it	104
24	Марина	Москва	it	104
25	Иван	Москва	it	120
31	Вероника	Москва	sales	96
32	Григорий	Самара	sales	96
33	Анна	Москва	sales	100

Оконная функция

Составим рейтинг сотрудников по размеру заработной платы:

Было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
24	Марина	it	104
23	Леонид	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

Стало

rank	name	department	salary
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
4	Вероника	sales	96
4	Григорий	sales	96
5	Ксения	it	90
6	Елена	it	84
7	Борис	hr	78
8	Дарья	hr	70

Обратите внимание — сотрудники с одинаковой зарплатой получили один и тот же ранг (Леонид и Марина, Вероника и Григорий).
Как перейти от «было» к «стало»?

Сначала отсортируем таблицу по убыванию зарплаты:

```sql
select
  null as rank,
  name, department, salary
from employees
order by salary desc, id;
```

rank	name	department	salary
	Иван	it	120
	Леонид	it	104
	Марина	it	104
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96
	Ксения	it	90
	Елена	it	84
	Борис	hr	78
	Дарья	hr	70

Теперь пройдем от первой строчки до последней и проставим ранг каждой записи. Начнем с 1 и будем увеличивать ранг каждый раз, когда значение **salary** меньше, чем у предыдущей записи:

Чтобы проставить ранг, достаточно на каждом шаге смотреть только на значения из столбца **salary**. Назовем эти значения *окном*.

Попробуем описать содержимое *окна* словами:

1. Это значения столбца **salary**.
2. Они упорядочены от большего значения к меньшему.

Сформулируем то же самое на SQL:

window w as (order by salary desc)

window — ключевое слово, которое показывает, что дальше будет определение окна;
w — название окна (может быть любым);
(order by salary desc) — описание окна («значения столбца **salary**, упорядоченные по убыванию»).

Задача — посчитать ранг по окну **w**. На SQL это записывается как **dense_rank() over w**.

dense_rank() — это оконная функция, которая считает ранг по указанному окну. Логика **dense_rank()** такая же, как была у нас при ручном подсчете — начать с 1 и увеличивать ранг каждый раз, когда очередное значение из окна отличается от предыдущего.

Добавим окно и оконную функцию в исходный запрос:

Реализация запроса в MySQL:

```
select
  dense_rank() over w as rank,
  name, department, salary
from employees
window w as (order by salary desc)
order by rank, id;
```

Реализация запроса в ClickHouse:

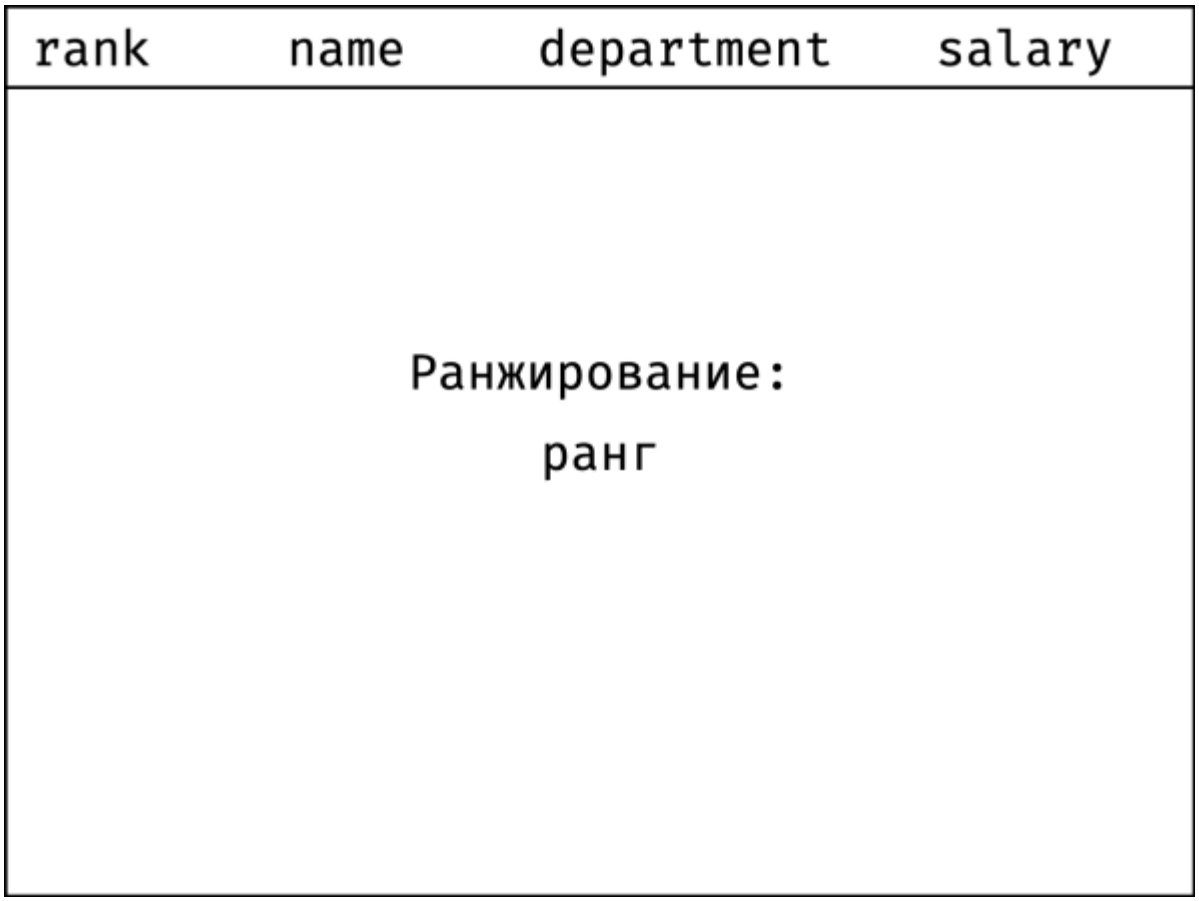
```
select
  dense_rank() over w as rank,
  name, department, salary
from employees
window w as (order by salary desc)
order by rank, id;
```

rank	name	department	salary
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Анна	sales	100
4	Вероника	sales	96
4	Григорий	sales	96
5	Ксения	it	90
6	Елена	it	84
7	Борис	hr	78
8	Дарья	hr	70

Вот как движок выполняет такой запрос:

1. Берет таблицу, указанную в **from**.
2. Выбирает из нее все записи.
3. Для каждой записи рассчитывает значение **dense_rank()** с помощью окна **w**.
4. Сортирует результат как указано в **order by**.

Вот как отрабатывает шаг 3, на котором назначается ранг:



Конструкция **window** сама по себе ничего не делает с результатами запроса. Она только определяет окно, которое можно использовать (или не использовать) в запросе. Если убрать вызов **dense_rank()**, запрос отработает, как будто нет никаких окон. Окно начинает работать только тогда, когда в **select** появляется оконная функция, которая его использует.

Оконные запросы в СУБД Oracle и MS SQL Server

Ни **Oracle**, ни **SQL Server** не поддерживают конструкцию **window**. Чтобы заставить работать оконный запрос в этих СУБД, перенесите описание окна внутрь инструкции **over**.

Не так:

```
select
  dense_rank() over w as rank,
  name, department, salary
from employees
window w as (order by salary desc)
order by rank, id;
```

А так:

```
select
  dense_rank() over (
    order by salary desc
  ) as rank,
  name, department, salary
from employees
order by rank, id;
```

Несколько окон

Что делать, если понадобится использовать в запросе несколько окон? Просто перечислите их через запятую в разделе **window**:

```
select ...
from ...
where ...
window
  w1 as (...),
  w2 as (...),
  w3 as (...)
;
```

Например, отранжируем сотрудников по зарплате в прямом и обратном порядке:

```
select
  dense_rank() over w1 as r_asc,
  dense_rank() over w2 as r_desc,
  name, salary
from employees
window
  w1 as (order by salary asc),
  w2 as (order by salary desc)
order by salary, id;
```

Результат запроса:

r_asc	r_desc	name	salary
1	8	Дарья	70
2	7	Борис	78
3	6	Елена	84
4	5	Ксения	90
5	4	Вероника	96
5	4	Григорий	96
6	3	Анна	100
7	2	Леонид	104
7	2	Марина	104
8	1	Иван	120

Секции окна

Теперь составим рейтинг сотрудников по размеру заработной платы независимо по каждому департаменту:

Было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
24	Марина	it	104
23	Леонид	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

Стало

rank	name	department	salary
1	Борис	hr	78
2	Дарья	hr	70
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Ксения	it	90
4	Елена	it	84
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96

Как перейти от «было» к «стало»?

Сначала отсортируем таблицу по департаментам, а внутри департамента — по убыванию зарплаты:

```sql
select
  null as rank,
  name, department, salary
from employees
order by department, salary desc, id;
```

rank	name	department	salary
	Борис	hr	78
	Дарья	hr	70
	Иван	it	120
	Леонид	it	104
	Марина	it	104
	Ксения	it	90
	Елена	it	84
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96

Теперь пройдем от первой строчки до последней и проставим ранг каждой записи. Начнем с 1 и будем увеличивать ранг каждый раз, когда значение **salary** меньше, чем у предыдущей записи. При переходе от департамента к департаменту будем сбрасывать ранг обратно на 1:

rank	name	depart ↑	salary ↓
1	Борис	hr	78
2	Дарья	hr	70
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
	Ксения	it	90
	Елена	it	84
	Анна	sales	100
	Вероника	sales	96
	Григорий	sales	96

и так далее...

Чтобы проставить ранг, достаточно на каждом шаге смотреть только на значения из столбца **salary**, выделенные синей рамкой. Это и есть окно в данном случае.

Видно, что окно меняется в зависимости от того, к какому департаменту относится текущая запись. Опишем словами:

1. Окно разбито на несколько независимых секций — по одной на департамент.
2. Внутри секции записи упорядочены по убыванию зарплаты.

Сформулируем то же самое на SQL:

window w as (partition by department order by salary desc)

- **partition by department** указывает, как следует разбить окно на секции;
- **order by salary desc** задает сортировку внутри секции.

Функция расчета ранга остается прежней — **dense_rank()**.

Добавим окно и оконную функцию в исходный запрос:

Реализация запроса в MySQL:

```sql
select
  dense_rank() over w as rank,
  name, department, salary
from employees
window w as (
  partition by department
  order by salary desc
)
order by department, rank, id;
```

Реализация запроса в ClickHouse:

```sql
select
  dense_rank() over w as rank,
  name, department, salary
from employees
window w as (
  partition by department
  order by salary desc
)
order by department, rank, id;
```

rank	name	department	salary
1	Борис	hr	78
2	Дарья	hr	70
1	Иван	it	120
2	Леонид	it	104
2	Марина	it	104
3	Ксения	it	90
4	Елена	it	84
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96

Вот как движок рассчитывает ранг для каждой записи:

rank	name	department	salary
		Ранжирование: ранг по секциям	

Группы

Разобьем сотрудников на три группы в зависимости от размера зарплаты:

- высокооплачиваемые,
- средние,
- низкооплачиваемые.

Было

id	name	department	salary
11	Дарья	hr	70
12	Борис	hr	78
21	Елена	it	84
22	Ксения	it	90
24	Марина	it	104
23	Леонид	it	104
25	Иван	it	120
31	Вероника	sales	96
32	Григорий	sales	96
33	Анна	sales	100

Стало

tile	name	department	salary
1	Иван	it	120
1	Леонид	it	104
1	Марина	it	104
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96
2	Ксения	it	90
3	Елена	it	84
3	Борис	hr	78
3	Дарья	hr	70

Как перейти от «было» к «стало»?

Сначала отсортируем таблицу по убыванию зарплаты.
Всего 10 записей, 3 группы — значит, две группы по 3 записи и одна 4 записи.
Чтобы провести границы между группами, придется анализировать все зарплаты, отсортированные по убыванию. Поэтому окно будет таким:

window w as (order by salary desc)

А вот функция потребуется другая — **ntile(n)**, где n — количество групп. В нашем случае:

Реализация запроса в MySQL:

```
select
  ntile(3) over w as tile,
  name, department, salary
from employees
window w as (order by salary desc)
order by salary desc, id;
```

Реализация запроса в ClickHouse:

```
select
  ntile(3) over w as tile,
  name, department, salary
from employees
window w as (
  order by salary desc
  rows between unbounded preceding and unbounded following
)
order by salary desc, id;
```

Результат запроса:

tile	name	department	salary
1	Иван	it	120
1	Леонид	it	104
1	Марина	it	104
1	Анна	sales	100
2	Вероника	sales	96
2	Григорий	sales	96
2	Ксения	it	90
3	Елена	it	84
3	Борис	hr	78
3	Дарья	hr	70

ntile(n) разбивает все записи на **n** групп и возвращает номер группы для каждой записи. Если общее количество записей (10 в нашем случае) не делится на размер группы (3), то первые группы будут крупнее последних.

ntile() всегда старается разбить данные так, чтобы группы были одинакового размера. Поэтому записи с одинаковым значением з/п вполне могут попасть в разные (соседние) группы:

```
select
  ntile(2) over w as tile,
  name, department, salary
from employees
window w as (order by salary desc, id)
order by salary desc, tile;
```

tile	name	department	salary	
1	Иван	it	120	
1	Леонид	it	104	
1	Марина	it	104	
1	Анна	sales	100	
1	Вероника	sales	96	<-- (!)
2	Григорий	sales	96	<-- (!)
2	Ксения	it	90	
2	Елена	it	84	
2	Борис	hr	78	
2	Дарья	hr	70	

Функции ранжирования

Оконные функции ранжирования:

Функция	Описание
row_number()	порядковый номер строки
dense_rank()	ранг строки
rank()	тоже ранг, но с пропусками
ntile(n)	разбивает все строки на n групп и возвращает номер группы, в которую попала строка

dense_rank() и **ntile()** мы уже разобрали.

row_number() нумерует строки в порядке, указанном в **order by**. Никаких неожиданностей.

rank() похож на **dense_rank()**, а разницу проще всего показать на примере.

```
select
  ••• over w as rank,
  name, department, salary
from employees
window w as (order by salary desc)
order by rank, id;
```

В одном случае вместо ••• укажем dense_rank(), а в другом — rank():

Реализация запроса в ClickHouse:

```
select
  rank() over w as rank,
  name, salary
from employees
window w as (order by salary desc)
order by rank, id;
```

dense_rank()

rank	name	salary	
1	Иван	120	
2	Леонид	104	
2	Марина	104	
3	Анна	100	(!)
4	Вероника	96	
4	Григорий	96	
5	Ксения	90	(!)
6	Елена	84	
7	Борис	78	
8	Дарья	70	

rank()

rank	name	salary	
1	Иван	120	
2	Леонид	104	
2	Марина	104	
4	Анна	100	(!)
5	Вероника	96	
5	Григорий	96	
7	Ксения	90	(!)
8	Елена	84	
9	Борис	78	
10	Дарья	70	

Реализация запроса в ClickHouse для row_number():

```
select
  row_number() over (order by salary desc, name, department) as row,
  name, department, salary
from employees
order by row
```

row	name	department	salary
1	Иван	it	120
2	Леонид	it	104
3	Марина	it	104
4	Анна	sales	100
5	Вероника	sales	96
6	Григорий	sales	96
7	Ксения	it	90
8	Елена	it	84
9	Борис	hr	78
10	Дарья	hr	70